B.Tech. Sem-V

Fundamentals of Operating System Design (CEUC301)

UNIT 3: CONCEPTUAL QUESTION BANK

Synchronization and Concurrency Control Design

Unit	Unit 3
Question type	Conceptual, analytical and scenario-based

Syllabus coverage: Race conditions and the critical-section problem; semaphores, mutexes, monitors and condition variables; memory consistency and visibility models; lock-based and lock-free designs.

A. Race Conditions and the Critical-Section Problem

1. Define a race condition. Why can a program produce different results when the same concurrent operations execute in different orders?
2. Explain how a read-modify-write operation can cause a race condition when multiple threads access shared state.
3. Why does the presence of a race condition depend on execution interleavings rather than simply on the number of threads?
4. What is a critical section? Identify the entry section, critical section, exit section and remainder section in the critical-section problem.
5. Explain the three classical requirements of a correct critical-section solution: mutual exclusion, progress and bounded waiting.
6. Can a solution guarantee mutual exclusion but still be an incorrect solution to the critical-section problem? Explain using progress or bounded waiting.
7. Distinguish deadlock, starvation and a race condition. Can a concurrent program exhibit more than one of these problems?
8. Why is protecting each shared variable separately not always sufficient to preserve a higher-level invariant involving several variables?
9. How can code that appears correct during testing fail under a different thread interleaving or processor architecture?
10. Discuss the role of atomic operations in implementing synchronization primitives. Why is an ordinary assignment not necessarily sufficient?

B. Semaphores, Mutexes, Monitors and Condition Variables

11. Explain the purpose of a semaphore and distinguish binary semaphores from counting semaphores.
12. How do the wait (P) and signal (V) operations coordinate access to shared resources?
13. Compare a mutex with a semaphore in terms of ownership, intended use and misuse risks.
14. Why should a mutex generally be released by the thread that acquired it? How does ownership help detect programming errors?
15. Explain how a counting semaphore can represent a pool of interchangeable resources.
16. What is a monitor, and how does it combine shared data with operations that enforce synchronized access?
17. How do condition variables allow a thread to wait for a state change without continuously polling shared state?
18. Why should a condition-variable wait generally be placed inside a loop that rechecks the associated predicate?

19. Explain the relationship between a condition variable and the mutex protecting the predicate on which a thread waits.
20. Compare semaphores and monitors in terms of abstraction, ease of reasoning and the kinds of synchronization errors they can help prevent.
21. What is a lost wake-up problem? Explain how incorrect coordination between checking a condition and waiting can cause it.
22. How can incorrect lock ordering lead to deadlock? Describe design practices that reduce this risk.

C. Memory Consistency and Visibility Models

23. Explain why mutual exclusion and memory visibility are related but distinct concerns in concurrent programming.
24. What does it mean for one thread's write to a shared variable to become visible to another thread?
25. Why can compiler optimizations and processor reordering make unsynchronized concurrent code behave unexpectedly?
26. Explain the happens-before relationship and its importance in reasoning about concurrent execution.
27. How can releasing a lock and subsequently acquiring the same lock establish ordering and visibility between threads?
28. Distinguish sequential consistency from weaker memory consistency models at a conceptual level.
29. Why might a program that appears correct on one processor fail on another when it relies on undocumented memory-ordering assumptions?
30. What are atomic variables and memory-ordering operations intended to guarantee, and what do they not automatically guarantee?
31. Why is declaring a shared variable as volatile not, by itself, a general replacement for mutexes or atomic synchronization?
32. How do language-level memory models help programmers write portable concurrent software across different compilers and hardware platforms?

D. Lock-Based and Lock-Free Designs

33. Explain the basic principle of lock-based synchronization and identify its main advantages.
34. What are lock contention and lock convoying? How can they reduce application throughput?
35. Compare spinlocks and blocking mutexes. Under what conditions might each be appropriate?
36. Why can holding a lock while performing slow I/O or calling unknown code be dangerous?
37. Define lock-free progress guarantees. How does lock-free differ from wait-free and obstruction-free progress?
38. Why does a lock-free algorithm not necessarily guarantee that every individual thread will make progress within a bounded time?
39. Explain the role of compare-and-swap (CAS) in constructing lock-free data structures.
40. What is the ABA problem in lock-free algorithms, and why can it make a successful compare-and-swap misleading?
41. Why can lock-free designs be difficult to implement, verify, debug and maintain compared with lock-based designs?
42. Compare lock-based and lock-free approaches in terms of contention, scalability, fairness, complexity and failure recovery.
43. Why does eliminating locks not automatically eliminate race conditions or guarantee better performance?

E. Integrative and Higher-Order Conceptual Questions

44. A shared counter occasionally reports an incorrect total when updated by multiple worker threads. Explain the likely cause and compare suitable synchronization remedies.

45. A producer-consumer system wastes CPU time because consumers repeatedly check whether data is available. Recommend a synchronization design and justify your choice.
46. A service uses several mutexes and occasionally freezes under load. Explain how you would investigate lock-ordering problems and reduce deadlock risk.
47. A concurrent program uses atomic variables but still produces inconsistent application-level results. Explain how individual atomic operations can fail to preserve a multi-step invariant.
48. A developer claims that a program is thread-safe because it passes tests on a multicore development machine. Critically evaluate the claim and propose a stronger validation strategy.
49. A high-throughput data structure must support many concurrent readers and writers. Explain the factors that should guide a choice between locks and a lock-free design.
50. A lock-free stack occasionally loses updates under contention. Identify conceptual failure modes that should be investigated, including atomicity, ABA and memory reclamation.
51. Design a conceptual checklist for reviewing a concurrent component. Include shared-state invariants, mutual exclusion, progress, visibility, deadlock and testing.
52. Critically assess the statement: 'Using atomic operations makes synchronization unnecessary.' Distinguish atomicity, ordering and coordination in your answer.

F. Software Solutions to the Critical-Section Problem: Peterson's Solution and Busy Waiting

53. Explain the assumptions under which Peterson's solution provides mutual exclusion for two processes or threads.
54. How do the shared variables flag and turn work together in Peterson's solution to ensure that both participants do not enter the critical section simultaneously?
55. Explain how Peterson's solution satisfies mutual exclusion, progress and bounded waiting under its classical assumptions.
56. Why is Peterson's solution primarily of theoretical and educational importance rather than a general-purpose synchronization primitive in modern systems?
57. What is busy waiting? Explain why a process that repeatedly checks a lock or shared variable may waste CPU cycles.
58. Compare busy waiting with blocking synchronization. Under what conditions might a spin-based approach still be reasonable?
59. How can compiler optimizations and modern memory models affect a naive implementation of Peterson's solution?
60. A two-thread program uses Peterson's algorithm but fails on a modern multicore processor. Identify the assumptions and memory-ordering issues that should be investigated.

G. Hardware Solutions: Interrupt Disabling and Atomic Instructions

61. Explain how disabling interrupts can be used to protect a critical section on a uniprocessor system.
62. Why is disabling interrupts generally unsuitable as a user-level synchronization technique and insufficient as a general solution on multiprocessor systems?
63. What risks arise if interrupts remain disabled for too long in a system that requires responsive I/O and timer handling?
64. Explain the Compare-and-Swap (CAS) instruction and how it can be used to implement an atomic lock acquisition operation.
65. Explain the Exchange instruction and how atomic exchange can support mutual exclusion.

66. Compare Peterson's software solution, interrupt disabling, CAS and Exchange in terms of assumptions, portability, scalability and busy waiting.
67. Why must the read, comparison and update in a lock-acquisition operation be atomic? What could happen if they were separate ordinary operations?
68. A lock implemented with CAS performs poorly under heavy contention. Explain why this can happen and identify possible design-level remedies.

H. Binary and Counting Semaphores; Mutexes

69. Differentiate binary and counting semaphores, including the resource-management situations for which each is suitable.
70. Compare a binary semaphore and a mutex. Why should they not always be treated as interchangeable abstractions?
71. Explain how a counting semaphore initialized to the number of available resource instances controls concurrent access to a resource pool.
72. What incorrect behaviours can occur if semaphore wait and signal operations are performed in the wrong order?
73. Why must semaphore operations that update the semaphore state be atomic?
74. How can semaphores be used for both mutual exclusion and event ordering? Explain the conceptual difference between these uses.
75. A program uses a semaphore to protect shared data but sometimes deadlocks. What mistakes in acquisition, release or control flow might explain the problem?

I. Producer–Consumer Problem Using Semaphores

76. Describe the bounded-buffer producer–consumer problem and identify the shared state that requires coordination.
77. Explain the roles of the mutex, empty and full semaphores in a conventional bounded-buffer solution.
78. Why should the producer wait for an available buffer slot before inserting an item, and why should the consumer wait for an available item before removing one?
79. What can go wrong if the producer updates the buffer before successfully reserving an empty slot?
80. Why is the order of semaphore operations important in preventing deadlock and preserving buffer correctness?
81. A producer–consumer implementation works with one producer and one consumer but fails with multiple producers and consumers. Which assumptions and synchronization boundaries should be reviewed?
82. How would you explain the difference between protecting the buffer's critical section and coordinating the availability of items and empty slots?

J. Monitors and Condition Variables: Design and Correct Usage

83. Explain how a monitor encapsulates shared data and operations while providing controlled mutual exclusion.
84. How can condition variables within a monitor allow a thread to wait until a buffer becomes non-empty or non-full?
85. Why should a condition-variable wait recheck its predicate in a loop after waking?
86. Compare Hoare-style and Mesa-style monitor signalling at a conceptual level, including the implications for rechecking conditions.
87. Why is a signal on a condition variable not equivalent to permanently storing an event for a future waiter?
88. How do monitors and condition variables help express producer–consumer coordination more clearly than manually managed semaphores?
89. Identify common errors in condition-variable use, including missed notifications, incorrect predicates and inconsistent mutex protection.

K. Readers–Writers Problem and Its Solutions

90. Describe the readers–writers problem and explain why multiple readers may access shared data concurrently while writers generally require exclusive access.
91. Compare a reader-preference solution with a writer-preference solution. What fairness trade-offs does each introduce?
92. How can a readers–writers lock improve concurrency when read operations greatly outnumber write operations?
93. Explain how reader preference can cause writer starvation, and describe a policy that can reduce this risk.
94. Why might strict writer preference cause readers to wait for long periods under a continuous stream of writers?
95. What is a fair readers–writers solution expected to guarantee, and how can fairness differ from maximum throughput?
96. A shared database is read frequently and updated occasionally. Explain which readers–writers policy might be appropriate and what workload measurements should inform the choice.
97. Compare semaphore-based and monitor-based approaches to the readers–writers problem in terms of clarity, correctness and risk of starvation.